# TECHNICAL DESIGN DOCUMENT

## Employee Benefits Management System (EBMS)

Pinequest S3 Ep1 Project 2026

| | |
|---|---|
| **Project Name** | Employee Benefits Management System |
| **Document Author** | Engineering Team |
| **Version** | 1.0 |
| **Stakeholders** | HR Lead, Tech Lead, CEO, Finance Manager, Engineering Lead |
| **Classification** | Confidential — Internal Use Only |

# 1. Overview

This document describes the architecture and implementation of the Employee Benefits Management System (EBMS) — a fullstack, performance-driven platform that manages benefit eligibility, benefit requests, and vendor contracts for all employees. The system integrates with the existing OKR Dashboard and HR/Attendance services, and removes the burden of manual eligibility tracking from HR personnel.

> 🎯 **Hackathon Goal**
>
> Deliver a working end-to-end prototype that automates the full benefits eligibility pipeline — from an employee's profile and performance data, through rule-based eligibility computation, to a real-time dashboard and digital contract acceptance flow.

## 1.1 Problem Statement

The organization currently manages employee benefits through a fragmented collection of spreadsheets, manual HR processes, and disconnected vendor portals. This creates three critical problems:

- Employees have no single, transparent view of which benefits they qualify for and why.
- HR teams cannot enforce business rules (probation restrictions, OKR compliance, punctuality thresholds) consistently or at scale.
- Benefit contracts with third-party vendors (e.g., PineFit gym) carry company-paid obligations that must be actively tracked but are currently managed manually.

The absence of an automated eligibility engine means benefit abuse goes undetected, under-utilized benefits go unclaimed, and employee trust erodes when decisions feel arbitrary.

## 1.2 Proposed Solution

EBMS listens to employee profiles and performance signals. When eligibility-affecting data changes, it recomputes each employee's benefit access against a configurable rules engine, updates the employee's dashboard in real time, and enforces contract acceptance before any benefit is activated. HR administrators receive a full control plane to configure rules, manage vendor contracts, and override decisions with a mandatory audit trail.

## 1.3 Scope

| In Scope | Out of Scope |
|---|---|
| Real-time benefit eligibility computation | Digital signature workflow (DocuSign etc.) |
| Employee self-service benefits dashboard | Payroll integration or direct disbursement |
| Performance-gated rules (OKR, attendance, probation) | Mobile native application |

| | |
|---|---|
| Vendor contract lifecycle management | Multi-tenant / multi-company support |
| Digital contract acceptance by employees | Real-time attendance API (v2) |
| HR admin rule configuration (no-code) | Legacy benefit record migration |
| Full audit trail for all eligibility decisions | Multi-language UI |
| All 11 company benefits (Gym, Insurance, MacBook, etc.) | |

# 2. Functional Requirements

This section defines what the system must do, derived directly from the stated business requirements. Each requirement is traceable to a specific stakeholder need.

## FR-01 — Employee Benefits Dashboard

> 📊 **Requirement**
>
> The system must provide each authenticated employee a real-time, personalized dashboard showing their full benefit portfolio with eligibility status, eligibility reasoning, and the ability to request benefits and review vendor contracts.

- Employees can browse all available benefits in the company catalog grouped by category: Wellness, Equipment, Financial, Career Development, Flexibility.
- Each benefit displays: name, description, subsidy percentage, vendor details, eligibility criteria summary, and active contract link.
- Each benefit shows one of four statuses: ACTIVE (currently receiving), ELIGIBLE (qualifies, not yet requested), LOCKED (disqualified by rule), PENDING (request under review).
- Locked benefits display a human-readable explanation of which rule is blocking them (e.g., "OKR not submitted for Q1 2025" or "Late arrival threshold exceeded — 3 arrivals this month").
- Employees can tap any benefit for a detailed eligibility breakdown showing all rules evaluated and their pass/fail status.

## FR-02 — Benefit Request Flow

> 📋 **Requirement**
>
> Eligible employees must be able to submit a benefit request directly from the dashboard. Requests that involve vendor-paid benefits must require digital contract acceptance before submission.

- Eligible employees can submit a benefit request from the dashboard with a single action.
- Requests for vendor-subsidized benefits (Gym, Insurance, MacBook, Travel) present the full current vendor contract PDF for digital acknowledgment before submission.
- Contract acceptance is logged with timestamp, employee ID, and contract version hash.
- Multi-stage requests requiring Finance Manager approval (e.g., Down Payment) route through a dedicated approval queue.
- Employees receive in-app and email notifications on all request status changes.

## FR-03 — Eligibility Engine

> ⚙️ **Requirement**
>
> The system must implement a rules-based eligibility engine that evaluates a configurable ruleset against each employee's live data profile and produces a deterministic eligibility decision with full audit trail for every benefit.

- Eligibility is computed as a set of rules joined by AND logic — all rules must pass for a benefit to be ELIGIBLE.
- Rules are configured per-benefit by HR administrators without code deployments.
- The engine re-evaluates eligibility on: login, OKR sync, attendance data import, HR manual override, and responsibility level change.
- All eligibility decisions are written to an immutable audit log with full rule evaluation trace.

## FR-04 — Performance-Gated Access Rules

> 🔒 **Requirement**
>
> Benefit access must be automatically gated by employee performance signals: probation status, OKR submission compliance, and punctuality thresholds that differ by role (teacher vs. non-teacher).

**Probation Rule:**

- Employees in probationary status are ineligible for: Gym (PineFit), Private Insurance, Down Payment, Extra Responsibility, MacBook subsidy, Remote Work, Travel.
- Employees on probation retain access to: Digital Wellness and Shit Happened Days (1 day max allocation).

**OKR Compliance Gate:**

- If an employee has not submitted OKR entries for the current quarter by the HR-configured deadline, all non-core benefits are locked.
- Core benefits (Digital Wellness, base Shit Happened Days) remain accessible regardless of OKR status.
- The system syncs with the OKR Dashboard via scheduled webhook; manual HR override is available.

**Attendance Punctuality Gate:**

- Teacher employees: late arrival = check-in after 09:00 AM. Three or more late arrivals in a rolling 30-day window triggers benefit restriction.
- Non-teacher employees: late arrival = check-in after 10:00 AM. Same 3-strike threshold in rolling 30-day window.
- Affected benefits when threshold exceeded: Gym (PineFit), Private Insurance, Remote Work.

## FR-05 — Benefit-to-Responsibility Mapping

**🔗 Requirement**

The system must implement a benefit mapping engine where each benefit is explicitly mapped to one or more eligibility rules including responsibility level, role, and tenure. Mappings must be configurable without code changes.

- Benefits are mapped to responsibility levels (1 = Standard, 2 = Senior, 3 = Lead/Management) configured by HR.
- The "Extra Responsibility" benefit is gated at responsibility level ≥ 2.
- The "UX Engineer" benefit (design tools, subscriptions) is gated to employees with role = 'ux_engineer'.
- If an employee is demoted or their role responsibility changes, benefits tied to higher tiers are automatically suspended.
- The mapping between benefits and rules is stored in a declarative configuration — adding or modifying rules requires only a config change, not a code deployment.

## FR-06 — Vendor Contract Management

**📄 Requirement**

All company-subsidized benefits involve third-party vendor contracts. The system must manage the full contract lifecycle: upload, versioning, employee acceptance, and expiry alerting.

- HR can upload and version vendor contracts (PDF) associated with each benefit via the admin panel.
- When an employee requests a subsidized benefit, the current active contract version is presented for digital acceptance.
- Employee acceptance is logged with timestamp, IP address, and contract version hash — creating a legally traceable record.
- HR receives alerts when vendor contracts are within 60 days of expiry.
- Contracts are stored in Cloudflare R2; employees access them via signed URLs (TTL = 7 days).
- Re-triggering a contract acceptance creates a new timestamped record — never overwrites existing acceptance logs.

## FR-07 — HR Administration Panel

**🛠️ Requirement**

HR administrators must have a dedicated control plane to view any employee's eligibility snapshot, configure rules, manage contracts, perform manual overrides with documented justification, and query the full audit log.

- HR can view any employee's full benefit eligibility snapshot with the computed rule evaluation breakdown.
- HR can manually override eligibility for any benefit with a mandatory reason (full audit log entry created).
- HR can grant temporary exceptions (e.g., medical leave exemption from attendance gate) with configurable expiry dates.
- HR can configure benefit eligibility rules through a UI without engineering involvement.
- Rule configuration changes are versioned and require a second HR admin approval before taking effect.
- HR can export audit logs by employee, date range, benefit type, or rule triggered.

## Functional Requirements Summary

| ID | Requirement | Priority | Hackathon MVP? |
|---|---|---|---|
| FR-01 | Employee Benefits Dashboard | Must Have | Yes |
| FR-02 | Benefit Request & Contract Acceptance Flow | Must Have | Yes |
| FR-03 | Rules-Based Eligibility Engine | Must Have | Yes |
| FR-04 | Performance-Gated Access Rules (Probation / OKR / Attendance) | Must Have | Yes |
| FR-05 | Benefit-to-Responsibility Mapping Engine | Must Have | Yes |
| FR-06 | Vendor Contract Lifecycle Management | Must Have | Yes |
| FR-07 | HR Administration Panel & Audit Trail | Should Have | Partial |

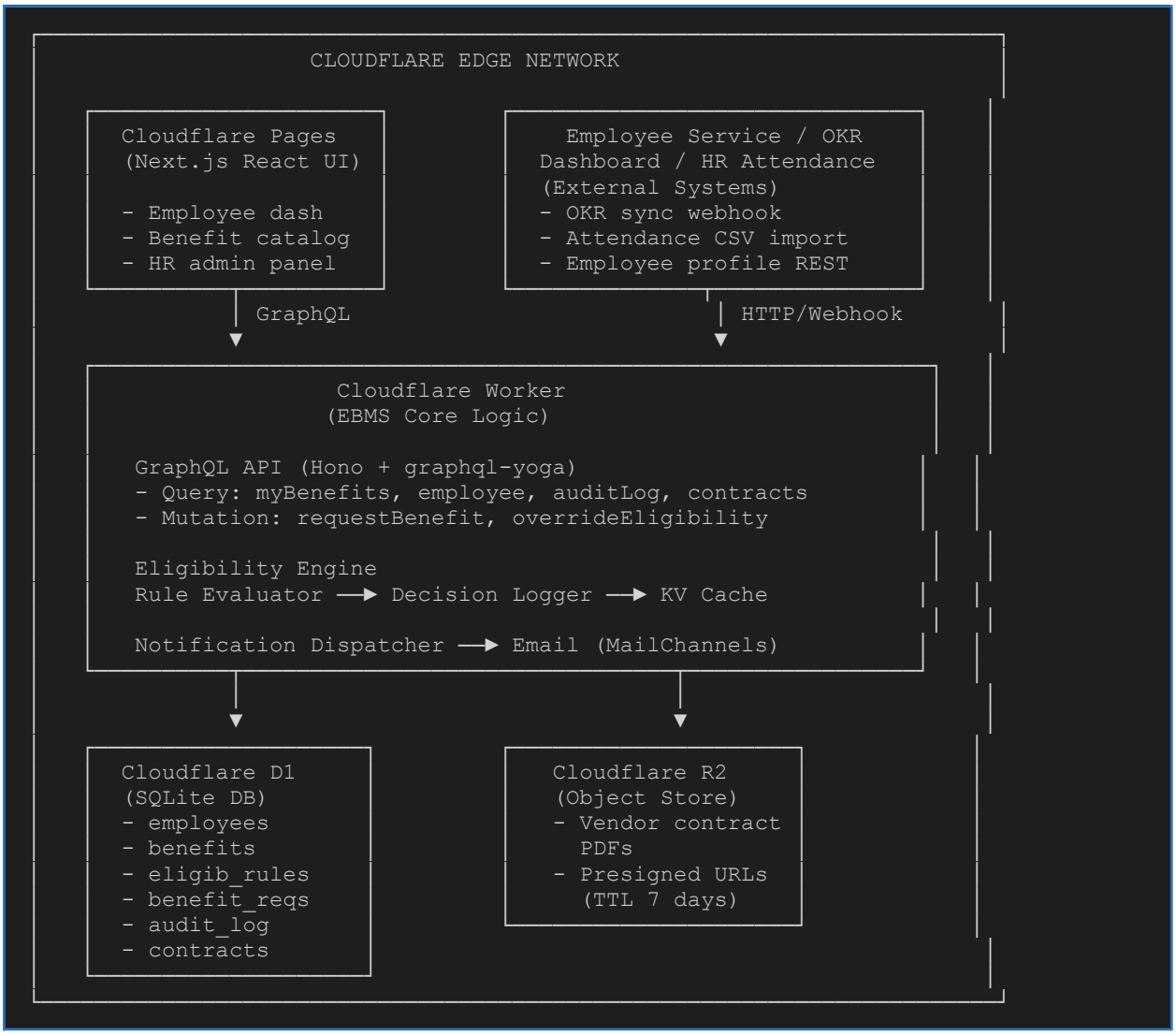
## 3. Glossary & Terminology

| Term | Definition |
|---|---|
| Benefit | A company-provided perk or subsidy available to eligible employees (e.g., Gym 50%, MacBook 50%). |
| Eligibility Engine | The rule evaluation service that determines benefit access per employee based on live data. |
| Eligibility Rule | A single configurable condition (e.g., employment_status == 'active') that gates a benefit. |
| OKR Gate | The rule that locks non-core benefits when an employee has not submitted their OKR for the current quarter. |
| Late Arrival | For teachers: check-in after 09:00. For non-teachers: check-in after 10:00. Three in 30 days triggers restriction. |
| Probation | Employment status for first 3 months or HR-flagged period; significantly limits benefit access. |
| Responsibility Level | Integer 1–3 representing seniority/scope of role; gates senior-tier benefits. |
| Shit Happened Day | Informal policy day off for unexpected personal circumstances; allocation varies by status. |
| Vendor Contract | A legal agreement with a third-party benefit provider (e.g., PineFit gym contract). |
| EBMS | Employee Benefits Management System — the system described in this TDD. |
| Trigger | An employee data change event that causes the eligibility engine to re-evaluate one or more benefits. |
| Override | An HR admin action that manually sets a benefit's eligibility status with mandatory documented justification. |

# 4. System Architecture

## 4.1 High-Level Architecture

EBMS follows an event-driven, edge-first architecture. It acts as a downstream consumer of HR and OKR data signals and orchestrates the full eligibility computation pipeline independently.




## 4.2 Component Breakdown

| Component | Responsibility | Technology |
|---|---|---|
| Employee Dashboard | Employee-facing UI: benefit catalog, eligibility status, request flow, contract acceptance. | Next.js 14 / Cloudflare Pages |

| Component | Responsibility | Technology |
|---|---|---|
| HR Admin Panel | HR control plane: rule config, overrides, contract upload, audit log viewer. | Next.js 14 / Cloudflare Pages |
| GraphQL API | Type-safe API layer between frontend and Worker. All queries and mutations. | Hono + graphql-yoga on Worker |
| Eligibility Engine | Evaluates rules per employee per benefit. Produces decision + rule trace. | TypeScript Worker module |
| OKR Sync Job | Daily Cron Trigger: fetches OKR submission flags, triggers re-evaluation. | Cloudflare Cron Trigger |
| Attendance Import | Ingests HR attendance CSV/API; computes late-arrival counts; updates employees. | Cloudflare Worker + D1 |
| Contract Store | Stores vendor contract PDFs; serves time-limited signed URLs. | Cloudflare R2 |
| Audit Logger | Immutable append-only log of all eligibility decisions, overrides, requests. | Cloudflare D1 (append-only) |
| Notification Dispatcher | Sends email notifications on request state changes and contract expiry alerts. | MailChannels / Resend |

# 5. Benefit Catalog & Eligibility Rules

The core of EBMS is a declarative eligibility mapping engine. Every benefit has a unique ID and is associated with one or more eligibility rules. This design allows reconfiguring the rule set for any benefit by editing a config — no code changes required.

## WELLNESS BENEFITS

### Benefit: Gym — PineFit (50% subsidy)

Triggered/Active when: Employee is active, OKR submitted, attendance threshold not exceeded, not on probation. Vendor contract (PineFit) acceptance required.

| Rule Type | Condition | Blocking Message |
|---|---|---|
| Employment Status | status == 'active' | Not available during probation or leave. |
| OKR Gate | okr_submitted == true | Submit your Q[current] OKR to unlock this benefit. |
| Attendance Gate | late_arrival_count < 3 | Attendance threshold exceeded this month. |
| Contract | Requires PineFit contract acceptance | — |

### Benefit: Private Insurance (50% subsidy)

| Rule Type | Condition | Blocking Message |
|---|---|---|
| Employment Status | status == 'active' | Not available during probation or leave. |
| OKR Gate | okr_submitted == true | Submit your Q[current] OKR to unlock this benefit. |
| Attendance Gate | late_arrival_count < 3 | Attendance threshold exceeded this month. |

### Benefit: Digital Wellness (100% — Core)

Core benefit. No performance gates. Available to all active employees regardless of OKR or attendance status.

| Rule Type | Condition | Blocking Message |
|---|---|---|
| Employment Status | status != 'terminated' | Not available after termination. |

## EQUIPMENT & CAREER BENEFITS

### Benefit: MacBook (50% subsidy)

| Rule Type | Condition | Blocking Message |
|---|---|---|
| Tenure | days_since_hire >= 180 | Available after 6 months of employment. |
| Employment Status | status == 'active' | Not available during probation or leave. |
| OKR Gate | okr_submitted == true | Submit your Q[current] OKR to unlock this benefit. |
| Responsibility Level | responsibility_level >= 1 | — |

### Benefit: Extra Responsibility

| Rule Type | Condition | Blocking Message |
|---|---|---|
| Employment Status | status == 'active' | Not available during probation. |
| OKR Gate | okr_submitted == true | OKR submission required. |
| Attendance Gate | late_arrival_count < 3 | Attendance threshold exceeded. |
| Responsibility Level | responsibility_level >= 2 | Requires Senior level (Level 2+). |

### Benefit: UX Engineer Tools (100%)

| Rule Type | Condition | Blocking Message |
|---|---|---|
| Role | role == 'ux_engineer' | Available to UX/Design role only. |
| Employment Status | status == 'active' | Active employment required. |
| OKR Gate | okr_submitted == true | OKR submission required. |

# FINANCIAL & FLEXIBILITY BENEFITS

## Benefit: Down Payment Assistance

| Rule Type | Condition | Blocking Message |
|---|---|---|
| Tenure | days_since_hire >= 730 | Available after 2 years of employment. |
| Employment Status | status == 'active' | Active employment required. |
| Responsibility Level | responsibility_level >= 2 | Requires Senior level (Level 2+). |
| OKR Gate | okr_submitted == true | OKR submission required. |
| Finance Approval | Requires Finance Manager review | Pending Finance approval. |

## Benefit: Shit Happened Days

| Rule Type | Condition | Blocking Message |
|---|---|---|
| Employment Status | Probation: 1 day max; Active: 3 days/year | Probation allocation: 1 day maximum. |
| OKR Gate | okr_submitted == true (for full allocation) | Submit OKR for full allocation. |

## Benefit: Remote Work

| Rule Type | Condition | Blocking Message |
|---|---|---|
| Employment Status | status == 'active' (not probation) | Not available during probation. |
| OKR Gate | okr_submitted == true | OKR submission required. |
| Attendance Gate | late_arrival_count < 3 | Attendance threshold exceeded. |

## Benefit: Travel (50% subsidy)

| Rule Type | Condition | Blocking Message |
|---|---|---|
| Tenure | days_since_hire >= 365 | Available after 12 months of employment. |
| Responsibility Level | responsibility_level >= 1 | — |
| OKR Gate | okr_submitted == true | OKR submission required. |
| Manager Pre-Approval | Manager must approve travel request | Awaiting manager pre-approval. |

### Benefit: Bonus Based on OKR

| Rule Type | Condition | Blocking Message |
|---|---|---|
| OKR Score | okr_submitted == true AND score >= threshold | OKR not submitted or score below threshold. |
| Attendance Gate | late_arrival_count < 3 | Attendance threshold exceeded. |
| Employment Status | status == 'active' | Active employment required. |
| Finance Approval | Requires Finance Manager processing | Pending Finance processing. |

# 6. Eligibility Rule Configuration Schema

Eligibility rules and their benefit mappings are stored in a versioned configuration. This is the single source of truth for the eligibility engine and can be updated by HR without a code deployment.

```
// eligibility-rules.json
{
  "benefits": {
    "gym_pinefit": {
      "name": "Gym — PineFit 50%",
      "category": "wellness",
      "subsidyPercent": 50,
      "requiresContract": true,
      "rules": [
        { "type": "employment_status", "operator": "eq", "value": "active",
          "errorMessage": "Not available during probation or leave." },
        { "type": "okr_submitted", "operator": "eq", "value": true,
          "errorMessage": "Submit your Q[current] OKR to unlock this benefit." },
        { "type": "attendance", "operator": "lt", "value": 3,
          "errorMessage": "Attendance threshold exceeded this month." }
      ]
    },
    "extra_responsibility": {
      "name": "Extra Responsibility",
      "category": "career",
      "rules": [
        { "type": "employment_status", "operator": "eq", "value": "active" },
        { "type": "okr_submitted", "operator": "eq", "value": true },
        { "type": "attendance", "operator": "lt", "value": 3 },
        { "type": "responsibility_level", "operator": "gte", "value": 2,
          "errorMessage": "Requires Senior level (Level 2+)." }
      ]
    }
    // ... all 11 benefits
  }
}
```

# 7. Employee Data Model & Field Mapping

## 7.1 Employee Fields Reference

| Field | Type | Used In / Purpose | Eligibility Trigger? |
|---|---|---|---|
| id | UUID | Primary key — all audit records | No |
| email | String | Auth identity; notification dispatch | No |
| name / nameEng | String | All dashboard display + contract PDFs | No |
| role | String | UX Engineer gate; role-based rules | YES → UX benefit gate |
| department | String | HR admin filtering; recipient routing | No |
| responsibility_level | Integer (1–3) | Extra Responsibility, Down Payment gates | YES → level-gated benefits |
| employment_status | Enum | Core eligibility gate for most benefits | YES → all gated benefits |
| hire_date | Date | Tenure calculations (MacBook, Travel, Down Payment) | YES → tenure gates |
| okr_submitted | Boolean | OKR Compliance Gate for all non-core benefits | YES → OKR gate |
| late_arrival_count | Integer | Attendance gate (rolling 30-day count) | YES → attendance gate |
| late_arrival_updated_at | Timestamp | Date of last attendance import | No |
| employee_code | String | All audit records and contract references | No |

## 7.2 Trigger Field Mapping — Eligibility Re-evaluation

| Employee Field Changed | Condition | Re-evaluation Triggered For |
|---|---|---|
| employment_status | Changes to 'probation', 'active', 'terminated' | All benefits |
| okr_submitted | Value changes (any) | All non-core benefits |
| late_arrival_count | Value changes (attendance import) | Gym, Insurance, Remote Work, Bonus |
| responsibility_level | Value changes (promotion or demotion) | Extra Responsibility, Down Payment, MacBook |
| hire_date | Set for first time (new employee) | All tenure-gated benefits |
| role | Value changes (role transfer) | UX Engineer benefit |
| imageUrl, email (non-trigger) | Value changes (any) | Silently ignored — no re-evaluation |

# 8. Data Flow & Sequence

### 📊 Eligibility Computation Flow — Example: Employee submits OKR

The sequence below shows how a single data change (OKR submission) propagates through the EBMS pipeline to update benefit eligibility for an employee.

```
1. OKR Dashboard: Employee submits Q2 OKR
   --> OKR system calls webhook: POST /api/v1/okr-sync
   --> Payload: { "employeeId": "emp-001", "okr_submitted": true, "quarter":
"Q2-2025" }

2. EPAS Worker — OKR Sync Handler:
   --> Validates JWT on incoming webhook
   --> UPDATE employees SET okr_submitted = 1 WHERE id = "emp-001"

3. Eligibility Engine — Re-evaluation trigger:
   --> Input: employeeId = "emp-001", changedField = "okr_submitted"
   --> Fetch all benefits with okr_submitted rule
   --> For each benefit: evaluate full rule chain

4. Rule Evaluator (per benefit — e.g., Gym / PineFit):
   Rule 1: employment_status == 'active'  → PASS
   Rule 2: okr_submitted == true          → PASS  ← changed from FAIL
   Rule 3: late_arrival_count < 3         → PASS
   Decision: ELIGIBLE (was LOCKED)

5. Audit Logger:
   INSERT INTO eligibility_audit (employeeId, benefitId, oldStatus, newStatus,
     ruleTrace, triggeredBy, computedAt)

6. KV Cache Invalidation:
   DELETE kv["eligibility:emp-001"]

7. Notification Dispatcher:
   --> Send email: 'Your Gym (PineFit) benefit is now ELIGIBLE!'

8. DONE. Eligibility updated and employee notified in < 5 seconds.
```

### 📋 Benefit Request Flow — Example: Employee requests Gym (PineFit)

The sequence below shows the benefit request pipeline including contract acceptance.

```
1. Employee: Clicks 'Request Benefit' on Gym (PineFit) card
   --> GraphQL Mutation: requestBenefit(benefitId: 'gym_pinefit')

2. Worker — Request Handler:
   --> Re-validate eligibility server-side (never trust client state)
   --> benefit.requiresContract == true → fetch active PineFit contract from D1
   --> Return: contractId, R2 signed URL (TTL 7 days), contract version hash
```

```
3. Frontend: Render contract PDF viewer
   --> Employee reads and accepts contract
   --> GraphQL Mutation: confirmBenefitRequest(requestId, contractAccepted: true)

4. Worker — Confirm Handler:
   --> Log: INSERT INTO benefit_requests (employeeId, benefitId, status='pending',
         contractVersionAccepted, contractAcceptedAt, ipAddress)
   --> INSERT INTO contract_acceptances (...)

5. Notification: HR receives email — 'New Gym benefit request from [employee]'

6. HR approves → benefit_requests.status = 'active'
   --> Employee notified: 'Your Gym (PineFit) benefit is now ACTIVE!'
```

# 9. GraphQL API Design

## 9.1 Schema Overview

```
# ─── Enums ──────────────────────────────────────────────────
enum BenefitStatus   { ACTIVE ELIGIBLE LOCKED PENDING }
enum RequestStatus   { PENDING APPROVED REJECTED CANCELLED }
enum EmploymentStatus { ACTIVE PROBATION LEAVE TERMINATED }

# ─── Core Types ─────────────────────────────────────────────
type Employee {
  id: ID!
  name: String!
  role: String!
  responsibilityLevel: Int!
  employmentStatus: EmploymentStatus!
  okrSubmitted: Boolean!
  lateArrivalCount: Int!
  benefits: [BenefitEligibility!]!
}

type BenefitEligibility {
  benefit: Benefit!
  status: BenefitStatus!
  ruleEvaluations: [RuleEvaluation!]!
  computedAt: String!
}

type RuleEvaluation {
  ruleType: String!
  passed: Boolean!
  reason: String!
}

type Benefit {
  id: ID!
  name: String!
  category: String!
  subsidyPercent: Int!
  requiresContract: Boolean!
  activeContract: Contract
}

# ─── Queries ────────────────────────────────────────────────
type Query {
  me: Employee!                              # Current employee profile
  myBenefits: [BenefitEligibility!]!        # Live eligibility for all benefits
  benefits(category: String): [Benefit!]!   # Benefit catalog
  employee(id: ID!): Employee                # HR only
  auditLog(filters: AuditFilters!): [AuditEntry!]!  # HR only
}

# ─── Mutations ──────────────────────────────────────────────
type Mutation {
  requestBenefit(input: BenefitRequestInput!): BenefitRequest!
  confirmBenefitRequest(requestId: ID!, contractAccepted: Boolean!):
BenefitRequest!
  cancelBenefitRequest(requestId: ID!): BenefitRequest!
  # HR Admin:
  overrideEligibility(input: OverrideInput!): BenefitEligibility!
```

```
  uploadContract(input: ContractInput!): Contract!
  reviewBenefitRequest(input: ReviewInput!): BenefitRequest!
}
```

## 9.2 Authorization Matrix

| Operation | Min Role Required | Notes |
|---|---|---|
| me, myBenefits, benefits | employee | Returns only own data. |
| requestBenefit, cancelBenefitRequest | employee | Eligibility re-validated server-side on every submission. |
| employee(id), employees, auditLog | hr_admin | Cannot query other employees as regular employee. |
| overrideEligibility, uploadContract | hr_admin | All changes immutably logged to audit table. |
| reviewBenefitRequest (financial) | finance_manager | Only for cost-impacting benefits (Down Payment, Bonus). |

# 10. Database Schema (Cloudflare D1 / Drizzle ORM)

All tables use SQLite-compatible types. No PostgreSQL-specific features (JSONB, CTEs with RETURNING). Designed for Cloudflare D1 via Drizzle ORM.

### employees

| Column | Type | Description |
|---|---|---|
| id | TEXT (UUID) | Primary key |
| email | TEXT UNIQUE | Corporate email; used for auth identity |
| name / name_eng | TEXT | Mongolian and English name fields |
| role | TEXT | e.g. 'teacher', 'engineer', 'ux_engineer', 'manager' |
| department | TEXT | Org unit for HR filtering and routing |
| responsibility_level | INTEGER (1–3) | 1 = Standard, 2 = Senior, 3 = Lead |
| employment_status | TEXT | 'active' \| 'probation' \| 'leave' \| 'terminated' |
| hire_date | TEXT (ISO8601) | Used for all tenure calculations |
| okr_submitted | INTEGER (0/1) | Current quarter OKR submission flag |
| late_arrival_count | INTEGER | Rolling 30-day late arrival count |
| late_arrival_updated_at | TEXT | Timestamp of last attendance import |
| created_at / updated_at | TEXT | Audit timestamps |

### benefits

| Column | Type | Description |
|---|---|---|
| id | TEXT (UUID) | Primary key |
| name | TEXT | e.g. 'Gym (PineFit) 50%', 'Private Insurance 50%' |
| category | TEXT | wellness / equipment / financial / career / flexibility |
| subsidy_percent | INTEGER | Company subsidy % (0–100) |
| vendor_name | TEXT | Third-party vendor — nullable for internal benefits |

| Column | Type | Description |
|---|---|---|
| requires_contract | INTEGER (0/1) | Whether vendor contract acceptance required on request |
| active_contract_id | TEXT FK | FK → contracts.id (nullable) |
| is_active | INTEGER (0/1) | Soft-delete / catalog visibility flag |

### eligibility_rules

| Column | Type | Description |
|---|---|---|
| id | TEXT (UUID) | Primary key |
| benefit_id | TEXT FK | FK → benefits.id |
| rule_type | TEXT | 'employment_status' \| 'okr_submitted' \| 'attendance' \| 'responsibility_level' \| 'role' \| 'tenure_days' |
| operator | TEXT | 'eq' \| 'neq' \| 'gte' \| 'lte' \| 'in' \| 'not_in' |
| value | TEXT (JSON) | Expected value(s) for the rule condition |
| error_message | TEXT | Human-readable message shown when this rule blocks access |
| priority | INTEGER | Evaluation order; lower = evaluated first |
| is_active | INTEGER (0/1) | Toggle without deleting |

### benefit_eligibility (computed cache)

| Column | Type | Description |
|---|---|---|
| employee_id + benefit_id | TEXT FK (composite PK) | Unique per employee-benefit pair |
| status | TEXT | 'active' \| 'eligible' \| 'locked' \| 'pending' |
| rule_evaluation_json | TEXT (JSON) | Array of {rule_type, passed, reason} — full decision trace |
| computed_at | TEXT | Timestamp of last computation |
| override_by | TEXT FK | HR admin who overrode (nullable) |

| Column | Type | Description |
|---|---|---|
| override_reason | TEXT | Mandatory justification for manual overrides |
| override_expires_at | TEXT | Expiry for temporary exceptions (nullable) |

### benefit_requests

| Column | Type | Description |
|---|---|---|
| id | TEXT (UUID) | Primary key |
| employee_id | TEXT FK | Requesting employee |
| benefit_id | TEXT FK | Requested benefit |
| status | TEXT | 'pending' \| 'approved' \| 'rejected' \| 'cancelled' |
| contract_version_accepted | TEXT | Contract version hash at time of employee acceptance |
| contract_accepted_at | TEXT | Timestamp of contract acceptance |
| reviewed_by | TEXT FK | HR / Finance admin who approved or rejected |
| created_at / updated_at | TEXT | Audit timestamps |

### contracts

| Column | Type | Description |
|---|---|---|
| id | TEXT (UUID) | Primary key |
| benefit_id | TEXT FK | FK → benefits.id |
| vendor_name | TEXT | e.g. 'PineFit' |
| version | TEXT | Semantic version string e.g. '2025.1' |
| r2_object_key | TEXT | Cloudflare R2 storage key for the contract PDF |
| sha256_hash | TEXT | Integrity hash of contract PDF for tamper detection |
| effective_date | TEXT | Contract start date |
| expiry_date | TEXT | Triggers 60-day expiry alert to HR |

| Column | Type | Description |
|---|---|---|
| is_active | INTEGER (0/1) | Only one active contract per benefit at any time |

# 11. Storage Architecture (Cloudflare R2)

## 11.1 R2 Object Path Structure

```
// Cloudflare R2 bucket: company-ebms-contracts

contracts/
  ├── gym_pinefit/
  │   ├── 2025.1/
  │   │   └── pinefit_contract_2025v1.pdf          ← active contract
  │   └── 2024.3/
  │       └── pinefit_contract_2024v3.pdf          ← archived
  ├── private_insurance/
  │   └── 2025.1/
  │       └── insurance_contract_2025v1.pdf
  └── macbook/
      └── 2025.1/
          └── apple_macbook_agreement_2025v1.pdf

// Signed URL pattern (TTL = 7 days):
//
https://r2.company.com/contracts/{benefitId}/{version}/{filename}?X-Expiry={ts}&X-Sig={sig}
```

## 11.2 Storage Backend Rationale

| Option | Pros | Cons | Decision |
|---|---|---|---|
| Cloudflare R2 | Zero egress fees; edge-native; S3-compatible; fine-grained signed URLs; integrated with Workers SDK. | Less familiar to HR than Google Drive. | PRIMARY — Mandatory |
| Google Drive API | HR familiarity; shareable links; folder UI. | Quota limits; slower than R2; requires OAuth; not edge-native. | Secondary / HR viewing only (v2 sync option) |

# 12. Non-Functional Requirements

| Category | Requirement | Target |
|---|---|---|
| Performance | Eligibility computation latency (per employee, all benefits) | < 500ms |
| Performance | Benefit dashboard load (cold) | < 2 seconds |
| Performance | OKR sync to eligibility update (end-to-end) | < 5 seconds |
| Reliability | Eligibility computation accuracy | > 99.5% |
| Reliability | Failed OKR/attendance sync retries | Max 3 retries (exponential backoff) |
| Security | Contract PDF download links | Signed URLs, TTL = 7 days |
| Security | Employee PII must not appear in plain-text logs | PII masking required |
| Auditability | Every eligibility decision must be persisted | Full immutable audit trail |
| Scalability | Handle concurrent eligibility re-evaluations (batch OKR sync) | 200+ employees / minute |
| Availability | Service uptime target | 99.9% monthly (Cloudflare SLA) |
| Compliance | Audit log retention | Minimum 5 years |

# 13. Technology Stack

The following stack is defined for EBMS. Mandatory technologies are fixed requirements. Recommended technologies are the engineering team's suggested choices to complement the mandatory stack — the team may substitute alternatives where justified.

## 13.1 Mandatory Technologies

> 🚨 **Locked Requirements**
>
> The four technologies below are non-negotiable. All architecture and implementation decisions must be made within these constraints.

| Technology | Role in EBMS | Why Mandatory |
|---|---|---|
| Cloudflare D1 | Relational database (SQLite at the edge) | Stores eligibility decisions, rules config, audit log, benefit requests, and contract metadata. Serverless-native, zero infrastructure to manage, globally distributed reads. |
| Cloudflare R2 | Object storage for vendor contract PDFs | Stores all vendor contract PDFs with structured paths per benefit/version. Zero egress fees. Presigned URLs with TTL enforcement. Replaces manual file management. |
| Cloudflare Pages & Workers | Application runtime and hosting platform | All backend logic (eligibility engine, GraphQL API, OKR/attendance sync, notification dispatch) runs as Cloudflare Workers. Employee dashboard and HR admin panel hosted on Cloudflare Pages. |
| GraphQL | API layer between frontend and backend | EBMS exposes a GraphQL API for querying benefit eligibility, requesting benefits, browsing audit logs, and managing the rule registry. Chosen over REST for its relational query model suited to the nested eligibility data shape. |

## 13.2 Recommended Technologies

The following technologies are recommended by the engineering team to complete the stack. These may be substituted at the team's discretion as long as they integrate with the mandatory Cloudflare platform.

| Layer | Recommended | Rationale | Alternatives |
|---|---|---|---|
| Runtime Language | TypeScript (strict) | Type safety on Workers; shared types across frontend/API; strong Cloudflare Workers SDK support. | JavaScript |
| GraphQL Server | Hono + graphql-yoga | Lightweight, runs natively on Cloudflare Workers with zero cold-start overhead. | Apollo Server (heavier), Pothos |
| ORM / Query Builder | Drizzle ORM | Designed for edge runtimes; first-class Cloudflare D1 support; type-safe SQL; zero overhead. | Kysely, raw D1 SQL |
| Frontend Framework | Next.js 14 (App Router) | SSR + React Server Components for fast initial loads; Cloudflare Pages deployment. | Vue, SvelteKit, Astro |
| Styling | Tailwind CSS + shadcn/ui | Rapid UI development; accessible components; zero runtime CSS overhead. | Chakra UI, MUI |
| Validation | Zod | Runtime schema validation on all API inputs and rule config changes. | Yup, Valibot |
| Email Dispatch | MailChannels (via Worker) | Native Cloudflare Workers integration; free tier covers hackathon volume. | SendGrid, Resend, Postmark |
| Auth | Clerk / Auth.js (OIDC) | JWT-based sessions; role claim propagation to Workers; SSO-ready. | Cloudflare Access, custom JWT |
| CI/CD | GitHub Actions + Wrangler CLI | Official Cloudflare deployment tool; integrates directly with Pages and Workers. | GitLab CI, Bitbucket Pipelines |
| Testing | Vitest + Playwright | Unit tests for eligibility engine rules; E2E tests for critical employee/HR flows. | Jest, Cypress |
| Observability | Cloudflare Analytics Engine + Sentry | Edge-native metrics + error tracking with source maps; eligibility failure alerting. | Datadog, New Relic |

## 13.3 Architecture Constraints from the Stack

- Cloudflare Workers have a 10ms CPU time limit on the free plan (50ms on paid). Eligibility rule evaluation is O(rules × benefits) — this is fast. Heavy operations (PDF rendering) must be offloaded to a separate service or pre-computed.
- Cloudflare D1 is SQLite-based. Schema design must avoid PostgreSQL-specific features (JSONB operators, CTEs with RETURNING). Use standard SQLite-compatible SQL.
- Cloudflare R2 is S3-compatible. All contract upload/download logic must use the R2 SDK or S3-compatible presigned URLs.
- GraphQL mutations must be secured with JWT auth (Cloudflare Access or Clerk) to prevent unauthorized eligibility overrides or benefit requests.
- There is no persistent in-memory state between Worker invocations. All eligibility state must be read from D1 or KV on each request.
- KV is used as a short-TTL eligibility cache (5-minute TTL) to reduce D1 reads on dashboard load. KV is invalidated on any data change that affects eligibility.

## 14. Risks & Mitigations

| Risk | Probability | Impact | Mitigation |
|---|---|---|---|
| Rule mis-configuration locks all employees from a benefit | Low | High | Preview/diff UI for rule changes; staged rollout; second HR admin approval required; instant rollback via config versioning. |
| OKR dashboard API is unreliable or changes schema | Medium | High | Abstract sync behind adapter layer; fallback to last-known okr_submitted state; daily alert on sync failure. |
| Employees contest eligibility decisions (perceived unfairness) | High | Medium | Full rule trace visible to employee in dashboard; HR override mechanism with documented reason; appeal workflow in v2. |
| Attendance import format changes from HR system | Medium | Medium | Configurable column mapping in import pipeline; schema validation on ingestion with error alerts to HR before data is committed. |
| D1 audit log grows unboundedly over 5-year retention | Low | Medium | Archive audit entries older than 12 months to R2 cold storage via scheduled Cron Trigger. |
| Vendor contract PDF access via signed URL leaks to unauthorized party | Low | High | Signed URLs scoped to requesting employee's session; 7-day TTL enforced; access logged; contract hash verified on every access. |
| Duplicate OKR sync events fire twice (at-least-once delivery) | Medium | Medium | Idempotency key based on (employeeId + quarter + event_hash) prevents double eligibility re-evaluation. |

# 15. Implementation Plan (Hackathon Sprints)

| Phase | Tasks | Est. Duration |
|---|---|---|
| Phase 0 — Setup | Repo scaffold (monorepo), Cloudflare Workers + D1 + R2 setup, Drizzle schema migrations, auth provider integration, eligibility-rules.json baseline, CI/CD pipeline | 2 hours |
| Phase 1 — Core Engine | Eligibility engine (all 7 rule types), rule evaluator unit tests (Vitest), audit log write path, KV cache with invalidation | 4 hours |
| Phase 2 — GraphQL API | GraphQL schema (Pothos/Yoga on Hono), all queries and mutations, JWT authorization middleware, Drizzle resolvers | 4 hours |
| Phase 3 — Employee Dashboard | Benefits catalog UI, eligibility status badges, rule breakdown modal, benefit request flow, contract PDF viewer (Next.js + Pages) | 5 hours |
| Phase 4 — Contract & R2 | Contract upload to R2, presigned URL generation, employee digital acceptance flow, contract version tracking | 3 hours |
| Phase 5 — HR Admin Panel | Employee eligibility overview, manual override UI, rule configuration editor, contract upload, audit log viewer | 4 hours |
| Phase 6 — Integrations | OKR sync Cron Trigger, attendance CSV import, contract expiry alert emails (MailChannels), notification dispatcher | 3 hours |
| Phase 7 — Demo Polish | Security review, E2E tests (Playwright), performance profiling, demo seed data, hackathon presentation preparation | 3 hours |

> ⚡ **Hackathon Priority**
>
> For the demo, prioritize the full eligibility dashboard for a single employee (all 11 benefits, OKR gate + attendance gate demonstrated live). The HR admin override and contract acceptance flow should be demonstrated as second-priority. The remaining integrations (OKR sync Cron, attendance import) share the identical pipeline and can be shown as configuration-only additions to prove the extensibility of the eligibility engine.

# 16. Alternatives Considered

| Approach | Description | Why Not Chosen |
|---|---|---|
| Extend Existing HRMS | Add a custom benefits module to the current HR management system. | HRMS platforms don't support edge deployment; rule engine customization severely limited; vendor lock-in; no GraphQL API; cannot integrate with Cloudflare stack. |
| REST API instead of GraphQL | Standard REST endpoints for each resource and operation. | Benefit dashboard requires highly relational, nested data fetching (employee → benefits → eligibility → rules → contracts). REST would require multiple round trips or bespoke over-fetching endpoints. |
| AWS Lambda + RDS | Serverless with Lambda functions, API Gateway, and PostgreSQL on RDS. | Cold starts on Lambda degrade eligibility computation latency; RDS at-rest costs; more infrastructure to manage; does not leverage Cloudflare edge advantages. |
| No-code (Zapier + Airtable) | Wire everything with Zapier automations and an Airtable base. | Not flexible enough for the rule engine complexity; no audit trail; hard to version rules; costly at scale; no code-level testability. |
| Synchronous REST (no events) | HR system calls EBMS directly on each employee data change. | Tight coupling; HR system must wait for eligibility computation; single point of failure; no replay capability on failure. |

# 17. Open Questions & Decisions Required

| # | Question | Owner | Status |
|---|---|---|---|
| 1 | What is the official definition of 'OKR submitted'? (Draft saved vs. submitted for review vs. scored by manager?) | Product + HR | Open |
| 2 | Does the attendance system expose an API, or is CSV export the only integration path? | Engineering + HR Systems | Open |
| 3 | For Shit Happened Days — is the annual allocation tracked in EBMS or in the existing leave management system? Integration needed? | HR Director | Open |
| 4 | Should employees be notified proactively when approaching the attendance threshold (2/3 late arrivals)? | Product | Open |
| 5 | Down Payment Assistance: is this a one-time benefit or can it recur? What is the maximum subsidy amount? | Finance Manager + HR | Open |
| 6 | Who signs off on rule configuration changes before going live? Single HR admin or dual approval? | HR Director | Decided: Dual approval |
| 7 | Should terminated employees retain read-only access to their historical benefit and contract acceptance records? | Legal + HR | Open |
| 8 | What prevents a non-trigger field update (e.g., profile photo) from accidentally firing an eligibility re-evaluation? | Engineering | Decided: Whitelist trigger fields only |

## Document Sign-Off

| Role | Name | Signature | Date |
|---|---|---|---|
| Author | | | |
| Tech Lead Reviewer | | | |

| Role | Name | Signature | Date |
|---|---|---|---|
| HR Lead Reviewer | | | |
| CTO Approval | | | |